Chapter 6
Project 2.1: Data Inspection Notebook

We often need to do an ad hoc inspection of source data. In particular, the very first time we acquire new data, we need to see the file to be sure it meets expectations. Additionally, debugging and problem-solving also benefit from ad hoc data inspections. This chapter will guide you through using a Jupyter notebook to survey data and find the structure and domains of the attributes.

The previous chapters have focused on a simple dataset where the data types look like obvious floating-point values. For such a trivial dataset, the inspection isn't going to be very complicated.

It can help to start with a trivial dataset and focus on the tools and how they work together. For this reason, we'll continue using relatively small datasets to let you learn about the tools without having the burden of **also** trying to understand the data.

This chapter's projects cover how to create and use a Jupyter notebook for data inspection. This permits tremendous flexibility, something often required when looking at new data for the first time. It's also essential when diagnosing problems with data that has — unexpectedly — changed.

A Jupyter notebook is inherently interactive and saves us from having to design and build an interactive application. Instead, we need to be

disciplined in using a notebook only to examine data, never to apply changes.

This chapter has one project, to build an inspection notebook. We'll start with a description of the notebook's purpose.

6.1 Description

When confronted with raw data acquired from a source application, database, or web API, it's prudent to inspect the data to be sure it really can be used for the desired analysis. It's common to find that data doesn't precisely match the given descriptions. It's also possible to discover that the metadata is out of date or incomplete.

The foundational principle behind this project is the following:

We don't always know what the actual data looks like.

Data may have errors because source applications have bugs. There could be "undocumented features," which are similar to bugs but have better explanations. There may have been actions made by users that have introduced new codes or status flags. For example, an application may have a "comments" field on an accounts-payable record, and accounting clerks may have invented their own set of coded values, which they put in the last few characters of this field. This defines a manual process outside the enterprise software. It's an essential business process that contains valuable data; it's not part of any software.

The general process for building a useful Jupyter notebook often proceeds through the following phases:

1. Start with a simple display of selected rows.
2. Then, show ranges for what appear to be numerical fields.
3. Later, in a separate analysis notebook, we can find central tendency (mean, median, and standard deviation) values after they've been cleaned.

Using a notebook moves us away from the previous chapters' focus on CLI applications. This is necessary because a notebook is interactive. It is designed to allow exploration with few constraints.

The **User Experience** (**UX**) has two general steps to it:

1. Run a data acquisition application. This is one of the CLI commands for projects in any of the previous chapters.
2. Start a Jupyter Lab server. This is a second CLI command to start the server. The `jupyter lab` command will launch a browser session. The rest of the work is done through the browser:
    1. Create a notebook by clicking the notebook icon.
    2. Load data by entering some Python code into a cell.
    3. Determine if the data is useful by creating cells to show the data and show properties of the data.

For more information on Jupyter, see **https://www.packtpub.com/product/learning-jupyter/9781785884870**.

6.1.1 About the source data

An essential ingredient here is that all of the data acquisition projects **must** produce output in a consistent format. We've suggested using NDJSON (sometimes called JSON NL) as a format for preserving the raw data. See ***Chapter 3***, ***Project 1.1: Data Acquisition Base Application***, for more information on the file format.

It's imperative to review the previous projects' acceptance test suites to be sure there is a test to confirm the output files have the correct, consistent format.

To recap the data flow, we've done the following:

- Read from some source. This includes files, RESTful APIs, HTML pages, and SQL databases.

- Preserved the raw data in an essentially textual form, stripping away any data type information that may have been imposed by a SQL database or RESTful JSON document.

The inspection step will look at the text versions of values in these files. Later projects, starting with ***Chapter 9***, ***Project 3.1: Data Cleaning Base Application***, will look at converting data from text into something more useful for analytic work.

An inspection notebook will often be required to do some data cleanup in order to show data problems. This will be enough cleanup to understand the data and no more. Later projects will expand the cleanup to cover all of the data problems.

In many data acquisition projects, it's unwise to attempt any data conversion before an initial inspection. This is because the data is highly variable and poorly documented. A disciplined, three-step approach separates acquisition and inspection from attempts at data conversion and processing.

We may find a wide variety of unexpected things in a data source. For example, a CSV file may have an unexpected header, leading to a row of bad data. Or, a CSV file may — sometimes — lack headers, forcing the acquire application to supply default headers. A file that's described as CSV may not have delimiters, but may have fixed-size text fields padded with spaces. There may be empty rows that can be ignored. There may be empty rows that delimit the useful data and separate it from footnotes or other non-data in the file. A ZIP archive may contain a surprising collection of irrelevant files in addition to the desired data file.

Perhaps one of the worst problems is trying to process files that are not prepared using a widely used character encoding such as UTF-8. Files encoded with CP-1252 encoding may have a few odd-looking characters when the decoder assumes it's UTF-8 encoding. Python's

`codecs` module provides a number of forms of alternative file encoding to handle this kind of problem. This problem seems rare; some organizations will note the encoding for text to prevent problems.

Inspection notebooks often start as `print()` functions in the data acquisition process to show what the data is. The idea here is to extend this concept a little and use an interactive notebook instead of `print()` to get a look at the data and see that it meets expectations.

Not all managers agree with taking time to build an inspection notebook. Often, this is a conflict between assumptions and reality with the following potential outcomes:

- A manager can assume there will be no surprises in the data; the data will be entirely as specified in a data contract or other schema definition.
  - When the data doesn't match expectations, a data inspection notebook will be a helpful part of the debugging effort.
  - In the unlikely event the data does match expectations, the data inspection notebook can be used to show that the data is valid.
- A manager can assume the data is unlikely to be correct. In this case, the data inspection notebook will be seen as useful for uncovering the inevitable problems.

Notebooks often start as `print()` or logger output to confirm the data is useful. This debugging output can be migrated to an informal notebook — at a low cost — and evolve into something more complete and focused on inspection and data quality assurance.

This initial project won't build a complicated notebook. The intent is to provide an **interactive** display of data, allowing exploration and investigation. In the next section, we'll outline an approach to this project, and to working with notebooks in general.

6.2 Approach

We'll take some guidance from the C4 model (**https://c4model.com**) when looking at our approach.

- **Context**: For this project, the context diagram has two use cases: acquire and inspect
- **Containers**: There's one container for the various applications: the user's personal computer
- **Components**: There are two significantly different collections of software components: the acquisition program and inspection notebooks
- **Code**: We'll touch on this to provide some suggested directions

A context diagram for this application is shown in ***Figure 6.1***.

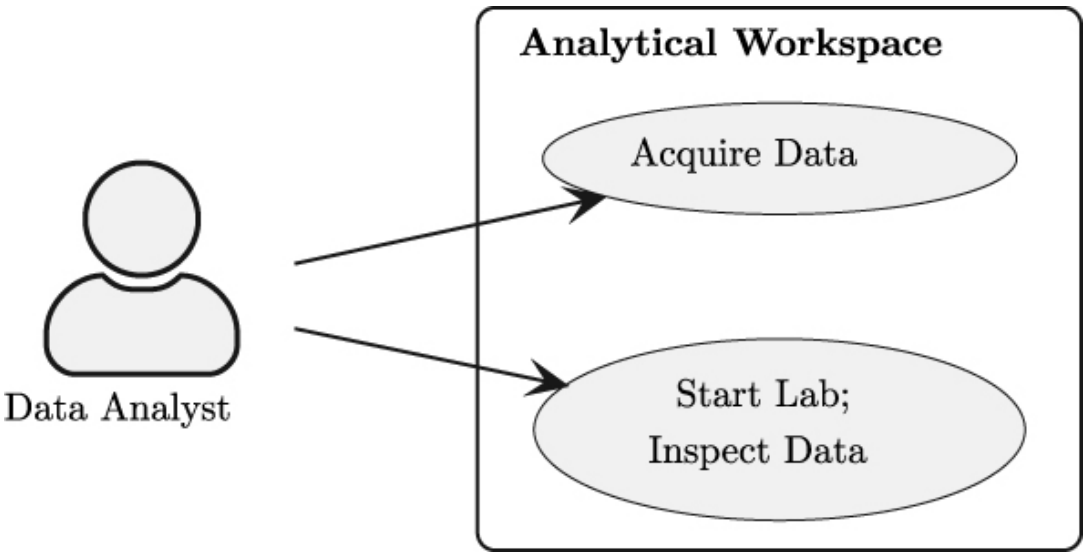
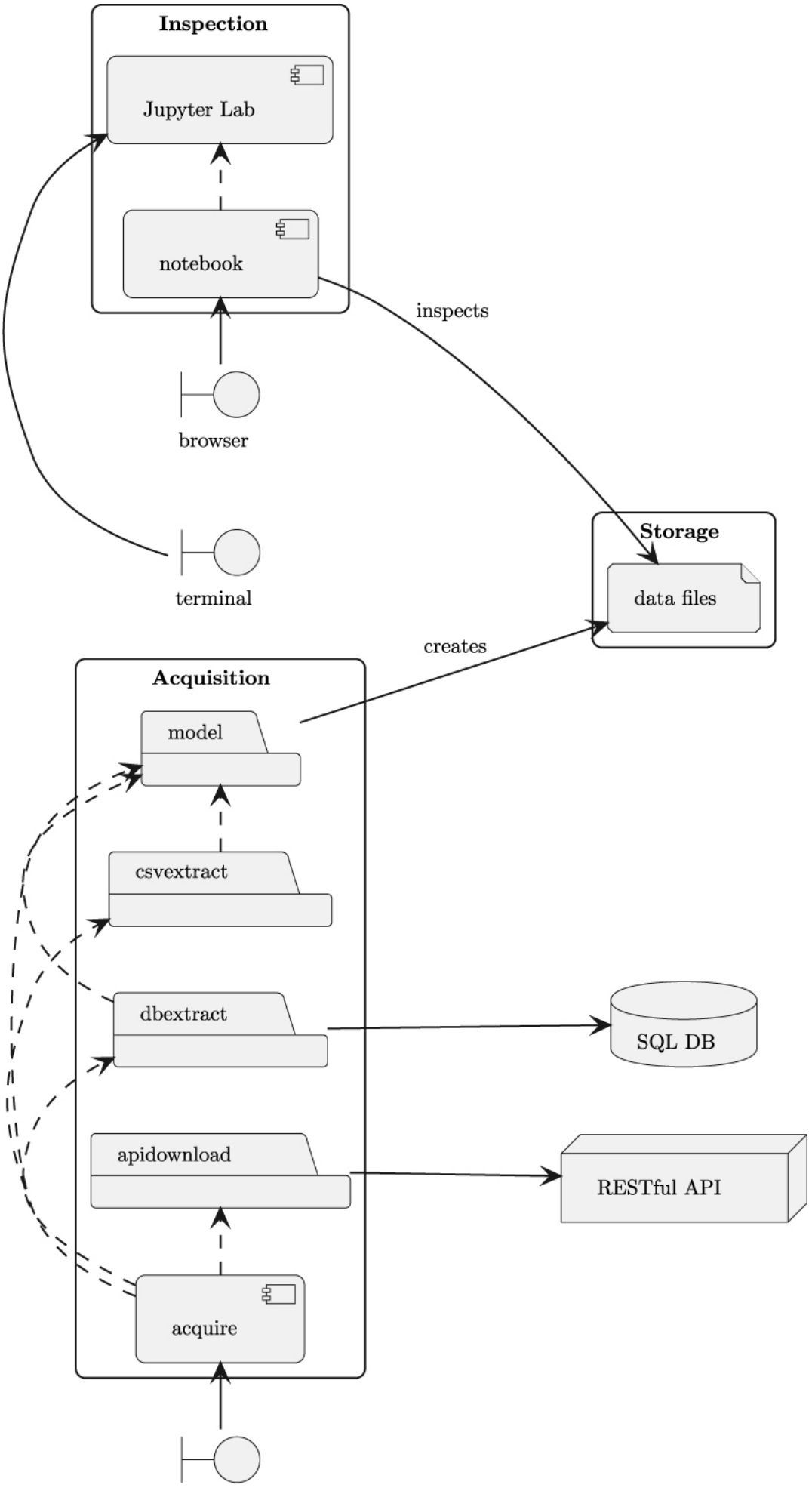


Figure 6.1: Context Diagram

The data analyst will use the CLI to run the data acquisition program. Then, the analyst will use the CLI to start a Jupyter Lab server. Using a browser, the analyst can then use Jupyter Lab to inspect the data.

The components fall into two overall categories. The component diagram is shown in ***Figure 6.2***.


Inspection
Jupyter Lab
notebook
inspects
browser
terminal
Storage
data files
creates
Acquisition
model
csvextract
dbextract
SQL DB
apidownload
RESTful API
acquire

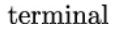


Figure 6.2: Component diagram

The diagram shows the interfaces seen by the data analyst, the `terminal` and the `browser`. These are shown with the *boundary* icon from the **Unified Modeling Language** (**UML**).

The `Acquisition` group of components contains the various modules and the overall acquire application. This is run from the command line to acquire the raw data from an appropriate source. The `db_extract` module is associated with an external SQL database. The `api_download` module is associated with an external RESTful API. Additional sources and processing modules could be added to this part of the diagram.

The processing performed by the `Acquisition` group of components creates the data files shown in the `Storage` group. This group depicts the raw data files acquired by the `acquire` application. These files will be refined and processed by further analytic applications.

The `Inspection` group shows the `jupyter` component. This is the entire Jupyter Lab application, summarized as a single icon. The `notebook` component is the notebook we'll build in this application. This notebook depends on Jupyter Lab.

The `browser` is shown with the boundary icon. The intention is to characterize the notebook interaction via the browser as the user experience.

The `notebook` component will use a number of built-in Python modules. This notebook's cells can be decomposed into two smaller kinds of components:

- Functions to gather data from acquisition files.

- Functions to show the raw data. The `collections.Counter` class is very handy for this.

You will need to locate (and install) a version of Jupyter Lab for this project. This needs to be added to the `requirements-dev.txt` file so other developers know to install it.

When using `conda` to manage virtual environments, the command might look like the following:

```
% conda install jupyterlab
```

When using other tools to manage virtual environments, the command might look like the following:

```
% python -m pip install jupyterlab
```

Once the `jupyter` products are installed, it must be started from the command line. This command will start the server and launch a browser window:

```
% jupyter lab
```

For information on using Jupyter Lab, see **https://jupyterlab.readthedocs.io/en/latest/**.

If you're not familiar with Jupyter, now is the time to use tutorials and learn the basics before moving on with this project.

Many of the notebook examples will include `import` statements.

Developers new to working with a Jupyter notebook should not take this as advice to repeat `import` statements in multiple cells throughout a notebook.

In a practical notebook, the imports can be collected together, often in a separate cell to introduce all of the needed packages.

In some enterprises, a startup script is used to provide a common set of imports for a number of closely related notebooks.

We'll return to more flexible ways to handle Python libraries from notebooks in ***Chapter 13***, ***Project 4.1: Visual Analysis Techniques***. There are two other important considerations for this project: the ability to write automated tests for a notebook and the interaction of Python modules and notebooks. We'll look at these topics in separate sections.

6.2.1 Notebook test cases for the functions

It's common to require unit test cases for a Python package. To be sure the test cases are meaningful, some enterprises insist the test cases exercise 100% of the lines of code in the module. For some industries, all logic paths must be tested. For more information, see ***Chapter 1***, ***Project Zero: A Template for Other Projects***.

For notebooks, automated testing can be a shade more complicated than it is for a Python module or package. The complication is that notebooks can contain arbitrary code that is not designed with testability in mind.

In order to have a disciplined, repeatable approach to creating notebooks, it's helpful to develop a notebook in a series of stages, evolving toward a notebook that supports automated testing.

A notebook is software, and without test cases, any software is untrustworthy. In rare cases, the notebook's code is simple enough that we can inspect it to develop some sense of its overall fitness for purpose. In most cases, complex computations, functions, and class definitions require a test case to demonstrate the code can be trusted to work properly.

The stages of notebook evolution often work as follows:

1. At stage zero, a notebook is often started with arbitrary Python code in cells and few or no function or class definitions. This is a great way to start development because the interactive nature of the notebook provides immediate results. Some cells will have errors or bad ideas in them. The order for processing the cells is not simply top to bottom. This code is difficult (or impossible) to validate with any automated testing.
2. Stage one will transform cell expressions into function and class definitions. This version of the notebook can also have cells with examples using the functions and classes. The order is closer to strictly top-to-bottom; there are fewer cells with known errors. The presence of examples serves as a basis for validating the notebook's processing, but an automated test isn't available.
3. Stage two has more robust tests, using formal `assert` statements or `doctest` comments to define a repeatable test procedure. Rerunning the notebook from the beginning after any change will validate the notebook by executing the `assert` statements. All the cells are valid and the notebook processing is strictly top to bottom.
4. When there is more complicated or reusable processing, it may be helpful to refactor the function and class definitions out of the notebook and into a module. The module will have a unit test module or may be tested via **doctest** examples. This new module will be imported by the notebook; the notebook is used more for the presentation of results than the development of new ideas.

One easy road to automated testing is to include **doctest** examples inside function and class definitions. For example, we might have a notebook cell that contains something like the following function definition:

```
def min_x(series: Series) -> float:
    """
    >>> s = [
```

```
...     {'x': '3', 'y': '4'},
...     {'x': '2', 'y': '3'},
...     {'x': '5', 'y': '6'}]
>>> min_x(s)
2
"""
return min(int(s['x']) for s in series.samples)
```

The lines in the function's docstring marked with `>>>` are spotted by the **doctest** tool. The lines are evaluated and the results are compared with the example from the docstring.

The last cell in the notebook can execute the `doctest.testmod()` function. This will examine all of the class and function definitions in the notebook, locate their **doctest** examples, and confirm the actual results match the expectations.

For additional tools to help with notebook testing, see **https://testbook.readthedocs.io/en/latest/**.

This evolution from a place for recording good ideas to an engineered solution is not trivially linear. There are often exploration and learning opportunities that lead to changes and shifts in focus. Using a notebook as a tool for tracking ideas, both good and bad, is common.

A notebook is also a tool for presenting a final, clear picture of whether or not the data is what the users expect. In this second use case, separating function and class definitions becomes more important. We'll look at this briefly in the next section.

6.2.2 Common code in a separate module

As we noted earlier, a notebook lets an idea evolve through several forms.

We might have a cell with the following

```
x_values = []
for s in source_data[1:]:
    x_values.append(float(s['x']))
min(x_values)
```

Note that this computation skips the first value in the series. This is because the source data has a header line that's read by the `csv.reader()` function. Switching to `csv.DictReader()` can politely skip this line, but also changes the result structure from a list of strings into a dictionary.

This computation of the minimum value can be restated as a function definition. Since it does three things — drops the first line, extracts the `'x'` attribute, and converts it into a float — it might be better to decompose it into three functions. It can be refactored again to include **doctest** examples in each function. See ***Notebook test cases for the functions*** for the example.

Later, this function can be cut from the notebook cell and pasted into a separate module. We'll assume the overall function was named `min_x()`. We might add this to a module named `series_stats.py`. The notebook can then import and use the function, leaving the definition as a sidebar detail:

```
from series_stats import min_x
```

When refactoring a notebook to a reusable module, it's important to use **cut and paste**, not copy and paste. A copy of the function will lead to questions if one of the copies is changed to improve performance or fix a problem and the other copy is left untouched. This is sometimes called the **Don't Repeat Yourself** (**DRY**) principle.

When working with external modules that are still under development, any changes to a module will require stopping the notebook kernel and rerunning the notebook from the very beginning to remove and reload the function definitions. This can become awkward. There

are some extensions to iPython that can be used to reload modules, or even auto-reload modules when the source module changes.

An alternative is to wait until a function or class seems mature and unlikely to change before refactoring the notebook to create a separate module. Often, this decision is made as part of creating a final presentation notebook to display useful results.

We can now look at the specific list of deliverables for this project.

6.3 Deliverables

This project has the following deliverables:

- A `pyproject.toml` file that identifies the tools used. For this book, we used `jupyterlab==3.5.3`. Note that while the book was being prepared for publication, version 4.0 was released. This ongoing evolution of components makes it important for you to find the latest version, not the version quoted here.
- Documentation in the `docs` folder.
- Unit tests for any new application modules in the `tests` folder.
- Any new application modules in the `src` folder with code to be used by the inspection notebook.
- A notebook to inspect the raw data acquired from any of the sources.

The project directory structure suggested in ***Chapter 1***, ***Project Zero: A Template for Other Projects*** mentions a `notebooks` directory. See ***List of deliverables*** for more information. Previous chapters haven't used any notebooks, so this directory might not have been created in the first place. For this project, the `snotebooks` directory is needed.

Let's look at a few of these deliverables in a little more detail.

6.3.1 Notebook .ipynb file

The notebook can (and should) be a mixture of Markdown cells providing notes and context, and computation cells showing the data.

Readers who have followed the projects up to this point will likely have a directory with NDJSON files that need to be read to construct useful Python objects. One good approach is to define a function to read lines from a file, and use `json.loads()` to transform the line of text into a small dictionary of useful data.

There's no compelling reason to use the `model` module's class definitions for this inspection. The class definitions can help to make the data somewhat more accessible.

The inspection process starts with cells that name the files, creating `Path` objects.

A function code like the following example might be helpful:

```
import csv
from collections.abc import Iterator
import json
from typing import TextIO

def samples_iter(source: TextIO) -> Iterator[dict[str, str]]:
    yield from (json.loads(line) for line in source)
```

This function will iterate over the acquired data. In many cases, we can use the iterator to scan through a large collection of samples, picking individual attribute values or some subset of the samples.

We can use the following statement to create a list-of-dictionary structure from the given path:

```
from pathlib import Path
source_path = Path("/path/to/quartet/Series_1.ndjson")
```

```
  with source_path.open() as source_file:
      source_data = list(samples_iter(source_file))
```

We can start with these basics in a few cells of the notebook. Given this foundation, further cells can explore the available data.

Cells and functions to analyze data

For this initial inspection project, the analysis requirements are small. The example datasets from the previous chapters are artificial data, designed to demonstrate the need to use graphical techniques for exploratory data analysis.

For other datasets, however, there may be a variety of odd or unusual problems.

For example, the **CO2 PPM — Trends in Atmospheric Carbon Dioxide** dataset, available at **https://datahub.io/core/co2-ppm**, has a number of "missing value" codes in the data. Here are two examples:

- The CO2 average values sometimes have values of −99.99 as a placeholder for a time when a measurement wasn't available. In these cases, a statistical process used data from adjacent months to interpolate the missing value.
- Additionally, the number of days of valid data for a month's summary wasn't recorded, and a −1 value is used.

This dataset requires a bit more care to be sure of the values in each column and what the columns mean.

Capturing the domain of values in a given column is helpful here. The `collections` module has a `Counter` object that's ideal for understanding the data in a specific column.

A cell can use a three-step computation to see the domain of values:

1. Use the `samples_iter()` function to yield the source documents.

2. Create a generator with sample attribute values.
3. Create a `Counter` to summarize the values.

This can lead to a cell in the notebook with the following statements:

```
from collections import Counter

values_x = (sample['x'] for sample in source_data)
domain_x = Counter(values_x)
```

The next cell in the notebook can display the value of the `domain_x` value. If the `csv.reader()` function is used, it will reveal the header along with the domain of values. If the `csv.DictReader()` class is used, this collection will not include the header. This permits a tidy exploration of the various attributes in the collection of samples.

An inspection notebook is not the place to attempt more sophisticated data analysis. Computing means or medians should only be done on cleaned data. We'll return to this in ***Chapter 15***, ***Project 5.1: Modeling Base Application***.

Cells with Markdown to explain things

It's very helpful to include cells using Markdown to provide information, insights, and lessons learned about the data.

For information on the markdown language, see the Daring Fireball website: **https://daringfireball.net/projects/markdown/basics**.

As noted earlier in this chapter, there are two general flavors of notebooks:

- **Exploratory**: These notebooks are a series of blog posts about the data and the process of exploring and inspecting the data. Cells may not all work because they're works in process.

- **Presentation**: These notebooks are a more polished, final report on data or problems. The paths that lead to dead ends should be pruned into summaries of the lessons learned.

A bright line separates these two flavors of notebooks. The distinguishing factor is the reproducibility of the notebook. A notebook that's useful for presentations can be run from beginning to end without manual intervention to fix problems or skip over cells with syntax errors or other problems. Otherwise, the notebook is part of an exploration. It's often necessary to copy and edit an exploratory notebook to create a derived notebook focused on presentation.

Generally, a notebook designed for a presentation uses Markdown cells to create a narrative flow that looks like any chapter of a book or article in a journal. We'll return to more formal reporting in ***Chapter 14***, ***Project 4.2: Creating Reports***.

Cells with test cases

Earlier, we introduced a `samples_iter()` function that lacked any unit tests or examples. It's considerably more helpful to provide a **doctest** string within a notebook:

```
def samples_iter(source: TextIO) -> Iterator[dict[str, str]]:
    """
    # Build NDJSON file with two lines
    >>> import json
    >>> from io import StringIO
    >>> source_data = [
    ...     {'x': 0, 'y': 42},
    ...     {'x': 1, 'y': 99},
    ... ]
    >>> source_text = [json.dumps(sample) for sample in source_data]
    >>> ndjson_file = StringIO('\\n'.join(source_text))

    # Parse the file
    >>> list(samples_iter(ndjson_file))
```

```
    [{'x': 0, 'y': 42}, {'x': 1, 'y': 99}]
    """
    yield from (json.loads(line) for line in source)
```

This function's docstrings include an extensive test case. The test case builds an NDJSON document from a list of two dictionaries. The test case then applies the `samples_iter()` function to parse the NDJSON file and recover the original two samples.

To execute this test, the notebook needs a cell to examine the docstrings in all of the functions and classes defined in the notebook:

```
import doctest
doctest.testmod()
```

This works because the global context for a notebook is treated like a module with a default name of `__main__`. This module will be examined by the `textmod()` function to find docstrings that look like they contain doc test examples.

Having the last cell run the **doctest** tool makes it easy to run the notebook, scroll to the end, and confirm the tests have all passed. This is an excellent form of validation.

6.3.2 Executing a notebook's test suite

A Jupyter notebook is inherently interactive. This makes an automated acceptance test of a notebook potentially challenging.

Fortunately, there's a command that executes a notebook to confirm it works all the way through without problems.

We can use the following command to execute a notebook to confirm that all the cells will execute without any errors:

```
% jupyter execute notebooks/example_2.ipynb
```

A notebook may ingest a great deal of data, making it very time-consuming to test the notebook as a whole. This can lead to using a cell to read a configuration file and using this information to use a subset of data for test purposes.

6.4 Summary

This chapter's project covered the basics of creating and using a Jupyter Lab notebook for data inspection. This permits tremendous flexibility, something often required when looking at new data for the first time.

We also looked at adding **doctest** examples to functions and running the **doctest** tool in the last cell of a notebook. This lets us validate that the code in the notebook is very likely to work properly.

Now that we've got an initial inspection notebook, we can start to consider the specific kinds of data being acquired. In the next chapter, we'll add features to this notebook.

6.5 Extras

Here are some ideas for you to add to this project.

6.5.1 Use pandas to examine data

A common tool for interactive data exploration is the `pandas` package.

See **https://pandas.pydata.org** for more information.

Also, see **https://www.packtpub.com/product/learning-pandas/9781783985128** for resources for learning more about pandas.

The value of using pandas for examining text may be limited. The real value of pandas is for doing more sophisticated statistical and graphical analysis of the data.

We encourage you to load NDJSON documents using pandas and do some preliminary investigation of the data values.